VibeSwap Maintenance Synthesis — 2026-04-27

Sources: three parallel audit agents dispatched 2026-04-27 against the VibeSwap codebase. **Scope**: contracts touched in the last 8 commits to master, with the dead-code audit extending across the full upgradeable-contract surface (`contracts/core/`, `contracts/libraries/`, `contracts/amm/`, `contracts/incentives/`, `contracts/governance/`, `contracts/messaging/`). **Purpose**: convert three separate audit reports into a single actionable PR roadmap. Items that are already fixed or verified correct are noted but not actioned. Items that are real are grouped into PRs ordered by ascending blast radius.

Executive summary

Three parallel agents — TRP-Solidity primitive audit, RSI strengthening review, dead-code audit — surfaced findings across the recently-committed surface plus the standing upgradeable contracts. The most important top-line:

- **Zero new critical or high-severity issues** introduced by recent commits. The Fibonacci cleanup pattern from `25940f97` is being repeated correctly elsewhere; `BatchMath.sol` is now a model of dead-code hygiene.
- **One real architectural gap surfaces from the TRP pass**: `VibeAMM.sol` wraps incentive-controller callbacks in `try/catch` blocks at five sites without a durable flag for failed callbacks or a permissionless retry path. This is a Settlement-State-Durability primitive violation. Funds are not at risk; incentive accounting can silently diverge from AMM state.
- **Two stale-storage upgrade hazards from the dead-code pass**: an unused `PIONEER_BONUS_MAX_BPS` constant in `ShapleyDistributor.sol` that a future implementation could mistake for the canonical cap, and two declared-but-never-reverted custom errors in `VibeAMM.sol`. Neither is exploitable today; both are misleading in ways that compound risk under upgrade.

- **A handful of trivial cleanup items** — comment rot from the Fibonacci rename, orphan helpers in `DeterministicShuffle`, alias constants and an orphan docstring in `CommitRevealAuction`, an event-emission asymmetry in `CircuitBreaker` — that bundle into a single low-controversy maintenance PR.
- **Two strengthening items** that are 1-line additions: an explicit threshold-ordering assertion in `FibonacciScaling.calculateRateLimit` (defends the scale-invariance invariant against future refactors) and a unit test for the recently-tuned `jarvis-bot` mention-bypass regex.

The recommended ship order is three PRs: a trivial cleanup, a strengthening pass, and an architectural Settlement-State-Durability fix that warrants design conversation before merge.

Section I — Already fixed or verified correct

These items appeared in the agent reports but require no action. Including them for completeness so the action items below are unambiguous.

Item	Status	Cite
Phantom Array Antipattern (`fibonacciWeightedPrice`, `calculateFibonacciClearingPrice`)	FIXED in `25940f97`	TRP #1, #2
Misleading function names (`applyGoldenRatioDamping`, `getFibonacciFeeMultiplier`)	RENAMED in `25940f97`	TRP #5
Settlement-Time Binding in `executeBatchSwap`	VERIFIED CORRECT	TRP #4 (VibeAMM:828-862)
Fibonacci window reset semantics	VERIFIED CORRECT (intentional)	TRP #6 (VibeAMM:1684-1688)
Analytics-only functions in `FibonacciScaling`	VERIFIED with explicit DO-NOT-PROMOTE warnings	TRP #8 (FibonacciScaling:235-414)

Item	Status	Cite
Fee-on-transfer token validation	VERIFIED CORRECT	TRP #9 (VibeAMM:1477-1482)
Withdrawal breaker small-LP exemption (CB-04)	VERIFIED CORRECT	TRP #10 (VibeAMM:696-713)
`getTierFeeMultiplier` NatSpec on φ-as-notation	VERIFIED CORRECT (already documented)	RSI #3
Premature abstraction (`fibonacciWeightedPrice`)	DELETED in `25940f97`	RSI #2
Dead code emergence in last 8 commits	NONE FOUND	RSI #7
Refactoring near-duplicates (`applyDeviationCap` call sites)	NOT REFACTORING CANDIDATE — variation is semantic	RSI #8
`BatchMath.applyDeviationCap` historical comment	POSITIVE — model of cleanup hygiene	DEAD-CODE #7

The pattern across these: the security and audit posture established by the Fibonacci cleanup commit is being maintained. The recent commits did not introduce regressions of the patterns the cleanup eliminated. This is a positive signal that the cleanup discipline is working as a forward-looking practice, not just a one-time event.

Section II — Real findings, grouped by recommended PR

PR 1 — Trivial cleanup bundle (low blast radius, ~30 min)

Theme: name-drift from rename refactors, dead artifacts, orphan stubs. None of these has any semantic impact. The bundle is mergeable as a single low-controversy PR.

1.1 — Stale `applyGoldenRatioDamping` reference in NatSpec - **Location**: `contracts/amm/VibeAMM.sol:2267` - **Description**: NatSpec on `_validateClearingPriceAgainstTruePrice` says "apply golden ratio damping" but the implementation now calls `applyDeviationCap` (renamed in `25940f97`). Comment-rot leftover from the rename pass. - **Fix**: change the NatSpec phrasing to "apply deviation cap." One-line edit. - **Source**: RSI #1.

1.2 — Remove declared-but-never-reverted custom errors from `VibeAMM` - **Location**: `contracts/amm/VibeAMM.sol:343-344` - **Description**: `error PriceImpactExceedsLimit(uint256 impact, uint256 maxAllowed);` and `error InsufficientPoolLiquidity(uint256 current, uint256 minimum);` are declared but never thrown anywhere in the codebase. Apparent remnants from an earlier design phase. - **Fix**: delete both error declarations. If a future upgrade plans to wire them up, that upgrade should re-introduce them with the corresponding `revert` sites. - **Source**: DEAD-CODE #3.

1.3 — Remove orphan docstring in `CommitRevealAuction` - **Location**: `contracts/core/CommitRevealAuction.sol:1480-1482` - **Description**: A multi-line `@notice` docstring starts at line 1480 but is never closed by an associated function — line 1483 begins the next function's docstring. Reads like documentation that was attached to a deleted function and never cleaned up. - **Fix**: delete lines 1480-1482. If the validation it described is load-bearing, the function that performs it should get a proper docstring as a separate edit. - **Source**: DEAD-CODE #6.

1.4 — Remove orphan helpers in `DeterministicShuffle` - **Location**: `contracts/libraries/DeterministicShuffle.sol:98-113, 122-140, 149-173` - **Description**: Three functions — `getShuffledIndex`, `verifyShuffle`, `partitionAndShuffle` — defined in the library but never called from any production code path. `verifyShuffle` is referenced only in tests. The core `shuffle()` and `generateSeed()` are live; these three are orphan helpers from earlier design iterations. - **Fix**: delete all three. Recoverable from git history if a future design needs them. The MEV-resistance argument depends on `shuffle()` only. - **Source**: DEAD-CODE #4.

1.5 — Remove `ATTENTION_WINDOW_COMMIT` / `ATTENTION_WINDOW_REVEAL` alias constants - **Location**: `contracts/core/CommitRevealAuction.sol:119-120` - **Description**: These are alias constants wrapping `COMMIT_DURATION` (8) and `REVEAL_DURATION` (2). The aliases exist to surface ETM-alignment naming intent in storage, but the actual protocol uses the unaliased constants throughout.

Notational duplication that risks divergence under future edits. - **Fix**: delete the aliases. If the ETM-alignment rationale must be surfaced, move it into inline comments next to `COMMIT_DURATION` and `REVEAL_DURATION` — naming intent belongs in comments, not in storage. - **Source**: DEAD-CODE #1.

1.6 — Mark `PIONEER_BONUS_MAX_BPS` as `@deprecated` - **Location**: `contracts/incentives/ShapleyDistributor.sol:91` - **Description**: This public constant declares a 5000 bps cap. The inline comment already states it is unused — the actual cap is hardcoded to 2.0× (10000 bps) in `_calculateWeightedContribution` (lines 788-795). Storage/ABI compatibility prevents outright deletion, but the constant's continued public visibility silently lies to any consumer who reads it. - **Fix**: add `@dev DEPRECATED — not used in current implementation. Actual pioneer multiplier cap is 2.0× hardcoded in _calculateWeightedContribution; this constant is retained for ABI compatibility only.` Move to a clearly-marked deprecated section. Consider whether ABI compatibility actually requires retention; if not, remove in a separate breaking-change PR. - **Source**: DEAD-CODE #2. This is the highest-risk item in this PR because of the upgrade-hazard profile (a future implementation could trust the constant as canonical when it is not).

PR 2 — Strengthening pass (low blast radius, additive only, ~1 hour)

Theme: defensive hardening that makes future refactors safer. All additive; no semantic changes.

2.1 — Add threshold-ordering invariant assertion in `calculateRateLimit` - **Location**: `contracts/libraries/FibonacciScaling.sol:204-233` - **Description**: The function applies damping at five thresholds (`FIB_236`, `FIB_382`, `FIB_500`, `FIB_618`, `FIB_786`). The scale-invariance property — the load-bearing argument that closes the timing-sweet-spot attack class — depends on these thresholds being strictly ordered. A future refactor that reorders the branches or changes the threshold values silently breaks the property. - **Fix**: at the top of the function, add `assert(FIB_236 < FIB_382 && FIB_382 < FIB_500 && FIB_500 < FIB_618 && FIB_618 < FIB_786);`. This costs zero gas at runtime (constants evaluated at compile time) and guarantees that any future edit that violates the ordering fails to compile. A small price for protecting the load-bearing invariant. - **Source**: RSI #5.

2.2 — Align `CircuitBreaker` event emission between reset paths - **Location**: `contracts/core/CircuitBreaker.sol:107, 110, 119-120, 246, 308` - **Description**: Manual reset (`disableBreaker()`, line 246) emits `BreakerDisabled`. Attested-resume (`_resumeAfterAttestation()`, line 308) emits `BreakerResumedByAttestation`. The two paths produce different event signatures for what is operationally the same state transition (breaker off → breaker on). Off-chain monitoring tools that watch one event miss the other; observability is incomplete. - **Fix**: emit both events from both paths (e.g., the attested-resume path emits `BreakerResumedByAttestation` *and* `BreakerDisabled` to maintain backward-compatible consumers, plus a new generic `BreakerStateChanged` event with a reason discriminator). Alternatively, refactor to a single `BreakerStateChanged(bytes32 indexed breakerType, bool active, uint8 reason)` event used by both paths. - **Source**: DEAD-CODE #5. The choice between additive (emit both) and refactor (single event) depends on whether the existing event consumers are external-stable; default to additive if uncertain.

2.3 — Add unit test for `jarvis-bot` mention-bypass regex - **Location**: `jarvis-bot/src/intelligence.js:189-195` (regex), `jarvis-bot/test/` (test target if exists) - **Description**: Commit `6659889b` added a mention-bypass regex `\b(jarvis|diablo)\b/i` that bypasses cooldown and confidence thresholds for messages mentioning the bot by name. There is no unit test covering the regex behavior — no test that "Jarvis can you explain X?" triggers the bypass; no test that "jarvis_like behavior" does NOT trigger (the word-boundary is load-bearing); no test that case-insensitivity works as intended. - **Fix**: add three test cases covering positive trigger, word-boundary negative, and case-insensitivity. If the test suite does not exist yet, document the three cases as a manual test plan in a `JARVIS_TESTING.md` adjacent to the source. - **Source**: RSI #6.

PR 3 — Architectural Settlement-State-Durability fix (medium blast radius, ~3-5 days)

Theme: closing a real gap in the incentive-controller callback flow. Worth design conversation before merge.

3.1 — Durable flag + permissionless retry on `VibeAMM` incentive callbacks - **Locations**: `contracts/amm/VibeAMM.sol:545, 656, 742, 1118, 1588` - **Description**: All five sites wrap calls to

`incentiveController.onLiquidityAdded()`, `onLiquidityRemoved()`, or `routeVolatilityFee()` in `try/catch` blocks of the form `try ... {} catch {}`. The try/catch correctly prevents an unhealthy incentive controller from bricking AMM operations — funds are never at risk from this gap. However, the implementation has no record of when a callback failed silently. The downstream consequence is that incentive accounting can diverge from AMM state in ways no one will notice until much later.

This is a Settlement-State-Durability primitive violation per the project's own primitive library. The primitive specifies: *async silent-catch needs durable flag + permissionless retry + downstream counter gate*. The current implementation has the silent-catch but lacks the other three components.

- **Fix**: introduce a durable record per (poolId, user, operationType) marking "callback pending"; emit an event when a callback fails; expose a permissionless `retryFailedCallback(poolId, user, operationType, payload)` function that any keeper can call to replay the callback; gate downstream incentive computations on the absence of pending callbacks for the relevant scope (or explicitly accept the divergence and document it).
- **Why this needs design conversation**: the fix adds storage (the failed-callback flags), adds at least one new external function, may add an event, and changes the operational semantics of callbacks (now visibly retryable rather than silently dropped). Storage layout changes need upgrade-compatibility review per the standing UUPS pattern. The choice of who can call the retry function (truly permissionless? gated to a keeper role? incentivized?) is a small mechanism-design question in itself.
- **Recommended approach**: open a draft PR with the smallest version of the fix (one storage mapping, one retry function, one event) and let the design discussion happen on the PR rather than in advance. The fix is well-scoped; the disagreement (if any) will be on details, not direction.
- **Source**: TRP #3.

PR 4 — Documentation (no code change, ~1 hour)

Theme: making forward operational hazards visible. No code touched.

4.1 — Write upgrade-path doc for the next `reinitializer(N)` cycle -
Description: The TRP pass flagged that VibeAMM's UUPS upgrade path is safe today (storage-only changes) but lacks documented patterns for the case

where the next upgrade adds new storage and needs to call an initializer. The OZ `reinitializer(N)` decorator is the right pattern but is not yet documented as the canonical approach for the protocol. - **Action**: write `docs/protocols/upgrade-path.md` (or similar) that documents: (a) the standing UUPS pattern, (b) when `reinitializer(N)` is required vs. not, (c) the pairing with `upgradeToAndCall` for atomic init+upgrade, (d) the version-numbering convention. - **Source**: TRP #7.

4.2 — Add CI lint rule against analytics-helper promotion - **Description**: The Fibonacci cleanup added explicit DO-NOT-PROMOTE-TO-STATE-MODIFYING-PATHS warnings on the analytics functions in `FibonacciScaling.sol`. These warnings are protective only if honored. A future refactor that copies `detectFibonacciLevel` or `calculateRetracementLevels` into a state-modifying path would create the predictable-trigger attack surface the warnings prevent. - **Action**: add a CI step that runs `grep -rn "detectFibonacciLevel\|calculateRetracementLevels\|goldenRatioMean\|calculateFibLiquidityScore\|calculatePriceBands" contracts/ | grep -v view | grep -v pure` and fails the build if any matches surface. Match against the actual list of analytics-marked functions. - **Source**: TRP #8.

Section III — Recommended ship order

The natural ordering is by ascending blast radius, which is also the ordering by audit-confidence margin:

Order	PR	Risk	Effort	Rationale
1	PR 1 (trivial cleanup)	Low	30 min	Mergeable in one sitting; no semantic change; clears clutter that compounds confusion.
2	PR 4 (documentation)	None	1 hour	Pure documentation; no code touched; protects future

Order	PR	Risk	Effort	Rationale
				operational decisions.
3	PR 2 (strengthening)	Low	1 hour	Additive only; no behavior change; protects load-bearing invariants against future drift.
4	PR 3 (architectural fix)	Medium	3-5 days	Real gap; needs design conversation; storage-layout-touching. Worth taking time on.

PRs 1 + 4 + 2 are all trivially safe to land as a sequence. PR 3 is the real work and should be opened as a draft for design feedback before merge.

Section IV — Forward-looking observations

A few patterns surfaced across the three audits that are worth naming explicitly because they suggest standing practices to keep, not just specific fixes to ship.

The Fibonacci cleanup is a template that's working. Three independent agents looked at the same codebase from three different angles and all of them concluded that the cleanup pattern (delete dead alternative implementations, rename misleading functions, mark analytics with DO-NOT-PROMOTE warnings, eliminate arithmetic-that-cancels-out) is being repeated correctly elsewhere. The dead-code agent found new candidates for the same treatment (`PIONEER_BONUS_MAX_BPS`, the `VibeAMM` dead errors). This suggests the cleanup discipline is generalizable, and that running this kind of audit periodically (every 3-6 months or after major commits) would catch the next batch.

The most dangerous remaining items have an "upgrade hazard" profile. Both `PIONEER_BONUS_MAX_BPS` and the dead `VibeAMM` errors are not exploitable today. Their risk is that *a future implementation* could mistake them for canonical when they are not. This is the upgradeable-contract amplification the dead-code agent specifically flagged: dead code is worse in upgradeable contracts because the next implementation can silently wire it up. The mitigation is the cleanup itself, plus a discipline of marking ABI-locked-but-stale items as `@deprecated` with an explicit pointer to the canonical source of truth.

Settlement-State-Durability is the underused primitive. PR 3 above closes one instance of the gap, but the primitive is general: any async/silent-catch path in the system warrants the (durable flag + permissionless retry + downstream counter gate) treatment. The five sites in `VibeAMM` are the most prominent instance, but a quick grep for `try ... catch {}` patterns elsewhere in the codebase would surface the next batch. Worth a future audit cycle scoped specifically to this primitive.

The codebase rewards structural hygiene. None of the three agents found a critical or high-severity issue introduced by the recent commits. The cleanup work is paying off in measurably tighter audit posture. Continue.

Synthesis authored by William Glynn with primitive-assist from JARVIS. Source agent reports archived in the agent-task-output cache for the 2026-04-27 session. The maintenance roadmap above is offered as a sequencing recommendation; specific PR composition can deviate from the proposed bundling without changing the underlying findings. PR 3 (Settlement-State-Durability) deserves the most thought; PRs 1, 2, and 4 should land soon.